Republic of the Philippines
CARAGA STATE UNIVERSITY
CABADBARAN CAMPUS
T. Curato St., Cabadbaran City, Agusan del Norte
Competence Service Uprightness

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Initializing the Flutter Web Project and First GitHub Checkpoint

Flutter Web Project Initialization and Source-Code Organization

Course	IT 110 - Web Systems and Technologies
Activity	Laboratory Activity No. 8
Topic	Flutter Web Project Initialization
Primary Output	Runnable Flutter Web starter project, structured lib/ directory, GitHub first commit, and evidence report
Maximum Score	80 points

1. Intended Learning Objectives

- Verify the Flutter/Dart development environment and initialize a Flutter Web project from the terminal.
- Launch the starter application in Google Chrome and record verifiable execution evidence.
- Organize the lib/ source into screens, widgets, services, models, routes, and constants with clearly assigned responsibilities.
- Create initial Home and Login pages (or equivalent approved starting screens) aligned with the group's Activity 6 UI flow.
- Explain the intended Flutter Web, REST API, and PostgreSQL technology stack without prematurely implementing the backend.
- Initialize Git version control, publish the first checkpoint to GitHub, and document how another group member can reproduce the run.
- Produce a concise, evidence-based setup report and orally defend the team's initialization decisions.

2. Activity Specifications

Item	Specification
Mode	Existing semester-project groups of three; each member must contribute and explain a responsibility.

Item	Specification
Suggested Time	One laboratory session (about 3 hours), plus scheduled demonstration / consultation.
Primary Output	Runnable Flutter Web starter project plus a public or instructor-accessible GitHub repository.
Documentary Output	Word or PDF report containing all five brief-required items and three authentic screenshots.
Presentation	5-7 minute group demonstration, then 2-3 minutes of questions; all group members participate.
Evidence Date	Original Activity 8 reference is dated October 12, 2026; confirm the actual submission deadline on the instructor's classroom announcement.

Technology stack - planned roles versus Activity 8 implementation

Layer / Tool	Role in the semester system	Status at Activity 8
Dart + Flutter Web	Client-side UI, widgets, navigation, forms, client logic	Initialize and run now
REST API (instructorapproved backend; e.g., FastAPI)	Business rules, endpoints, validation, server-side processing	Plan only; no API required
PostgreSQL (IT 108)	Persistent storage, entities and relational constraints	Plan/reference only
Git + GitHub	Version control, collaboration, checkpoint history	Initialize and push now
Chrome / local development server	Run and inspect the Flutter Web app	Demonstrate now

3. Create and Run the Flutter Web Application

Step 1 - Check the development environment

Open a terminal in your chosen workspace. Verify Flutter is available and that Chrome can be used as a web target. Resolve any blocking environment errors before proceeding.

```
flutter doctor flutter
devices
```

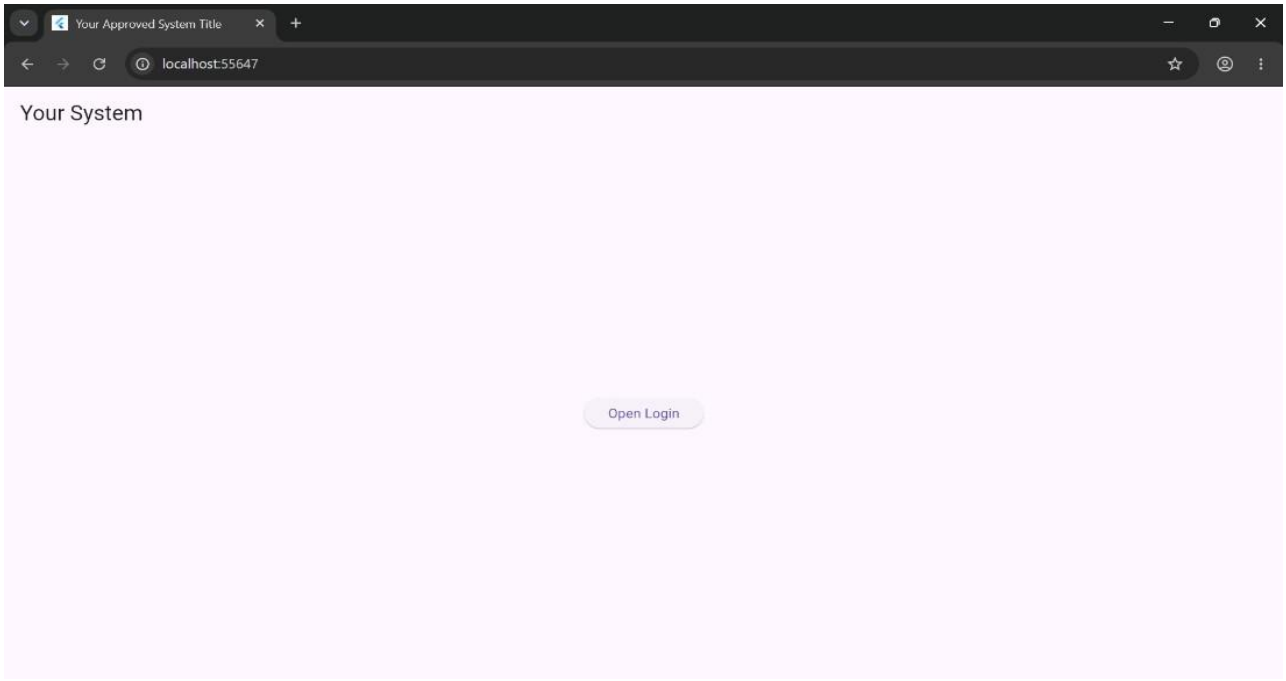
Step 2 - Create the project and open it

Follow the command sequence shown in Screenshot 1 of the uploaded activity brief. The name semester_project is the reference example; use that name unless the instructor approves another valid Dart project identifier.

```
flutter create semester_project
cd semester_project flutter run -d chrome
```

Expected result: Flutter generates the starter project, the development server starts, and Chrome opens a functioning Flutter Web application. The first compilation may take several minutes. Keep the running app visible while capturing the evidence.

Screenshot 1 - Mandatory evidence. Capture your terminal showing the project-creation/run command and the Flutter app visibly open in Chrome. A screenshot of typed commands without a successful launch is not sufficient.



If the browser target is not available

Use flutter devices to confirm available targets, check the Flutter SDK / Chrome installation, and consult the instructor. On SDK versions requiring web enablement, run flutter config --enable-web and restart the terminal. Use a second terminal for later commands while flutter run is active; Ctrl+C stops a running session.

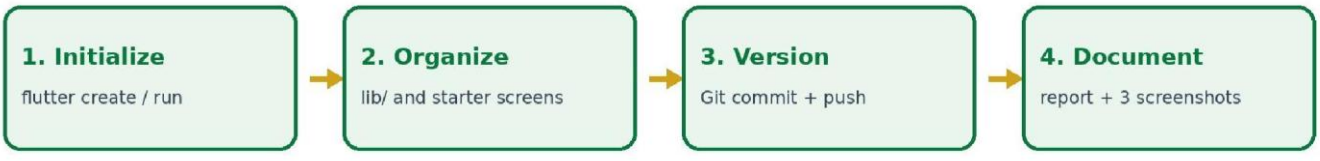


Figure 1. Activity 8 workflow: initialize, organize, version, and document the project baseline.

4. Organize the Flutter Source-Code Structure

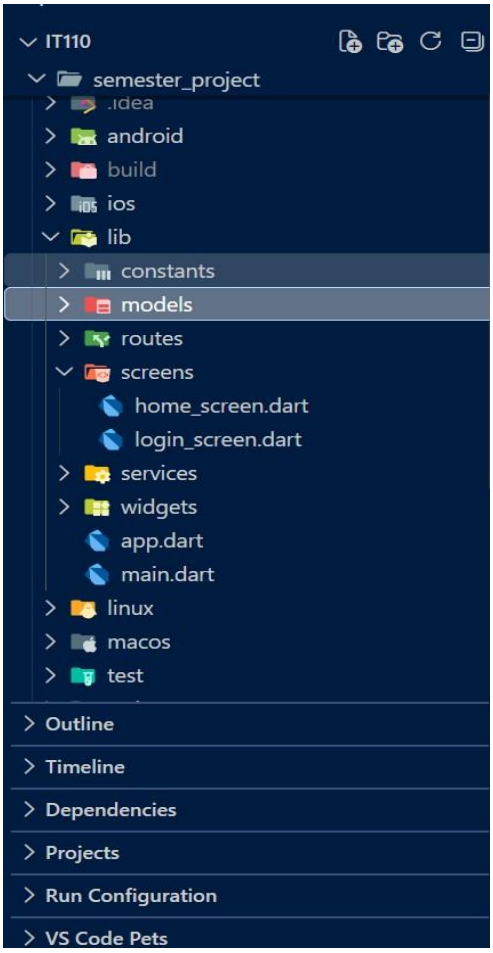
Using VS Code/Android Studio, create the folders and files in the example from Screenshot 2. Do not remove generated Flutter resources such as pubspec.yaml, web/, and the platform configuration files.

```
lib/
├── main.dart
├── app.dart
├── screens/
│   ├── home_screen.dart
│   └── login_screen.dart
├── widgets/
│   ├── app_button.dart
│   └── app_text_field.dart
├── services/
├── models/
├── routes/
└── constants/
```

Directory / File	Purpose in the group's future system	Your status
main.dart	Entry point; launches the root Flutter widget.	[ ]
app.dart	MaterialApp setup, theme, app-level navigation.	[ ]
screens/	User-facing pages from the approved Activity 6 UI flow.	[ ]
widgets/	Reusable UI components (buttons, inputs, cards).	[ ]
services/	Future API clients and external service integration.	[ ]
models/	Client-side data structures and JSON model classes.	[ ]
routes/	Routing definitions and navigation helpers.	[ ]
constants/	App-wide constants; never store secret API credentials.	[ ]

Important Git detail. Git does not track empty folders. Add placeholder .gitkeep files or a meaningful starter source file in services/, models/, routes/, and constants/ if you need these directories to appear in GitHub at the first checkpoint.

Screenshot 2 - Mandatory evidence. Capture the VS Code Explorer or equivalent file tree clearly showing lib/, main.dart, app.dart, the two starter screens, reusable widget files, and the supporting directories. Capture your terminal showing the project-creation/run command and the Flutter app visibly open in Chrome. A screenshot of typed commands without a successful launch is not sufficient.



5. Implement Initial Screens and a Reproducible Run

Create a simple running baseline, not the finished interface. At minimum, the Home screen and a Login placeholder (as shown in the reference structure) should be recognized in the code. Replace placeholder display text with the actual title of your approved system.

Minimal runnable reference - adapt to your own approved system

The following snippets illustrate how the starting files can work together. Teams may use equivalent implementations provided the project compiles and its navigation structure is understandable. **lib/main.dart**

```
import 'package:flutter/material.dart'; import
'app.dart';

void main() {   runApp(const
SemesterProjectApp());
}
```

lib/app.dart

```
import 'package:flutter/material.dart'; import
'screens/home_screen.dart'; import
'screens/login_screen.dart';                    class
SemesterProjectApp extends StatelessWidget {   const
SemesterProjectApp({super.key});

  @override
```

```
  Widget build(BuildContext context) => MaterialApp(
debugShowCheckedModeBanner: false,     title: 'Your
Approved  System  Title',        initialRoute:  '/',
routes: {        '/': (_) => const HomeScreen(),
      '/login': (_) => const LoginScreen(),
    },
  );
}
```

lib/screens/home_screen.dart

```
import 'package:flutter/material.dart';


class HomeScreen extends StatelessWidget {   const
HomeScreen({super.key});


  @override
  Widget build(BuildContext context) => Scaffold(      appBar:
AppBar(title: const Text('Your System')),     body:
Center(child: ElevatedButton(       onPressed: () =>
Navigator.pushNamed(context, '/login'),       child: const
Text('Open Login'),
    )),
  );
}
```

lib/screens/login_screen.dart

```
import 'package:flutter/material.dart';
class LoginScreen extends StatelessWidget
{   const LoginScreen({super.key});


  @override
  Widget build(BuildContext context) => Scaffold(
appBar: AppBar(title: const Text('Login')),      body:
const Center(child: Text('Login placeholder')),    );
}
```

6. Initialize Git and Publish the First Checkpoint

Step 1 - Check version control and commit the working baseline

Run these commands at the project root. The Activity 8 source shows git init, git add, git commit, git remote add, and git push. The branch-renaming line below is included to make the main branch consistent before the first push.

```
git init git add .  git commit -m "Initial
Flutter Web project" git branch -M main
```

Step 2 - Connect to the group repository and push

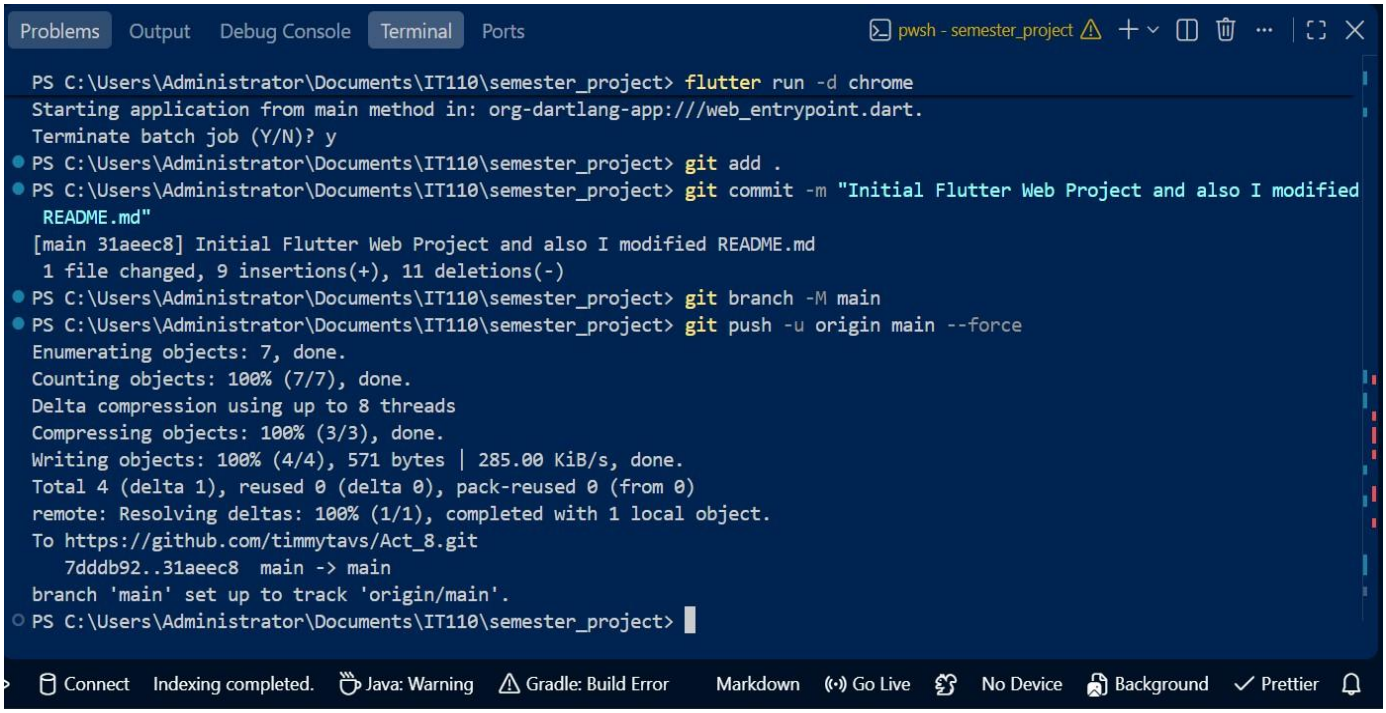
Create an empty GitHub repository first. Replace the example URL with your own team repository address. If an origin remote already exists, check git remote -v before adding or changing it.

```
git remote add origin <REMOTE-URL> git
```

```
push -u origin main
```

On GitHub, verify the files are visible, the main branch is selected, and the initial commit is present. Never commit .env files, access tokens, passwords, database credentials, or other private keys. Respect the generated .gitignore entries.

Screenshot 3 - Mandatory evidence. Capture the relevant Git commands and a successful push result, then confirm the first commit on GitHub. If the push fails, document the issue and fix it before claiming completion.



Required screenshot evidence - each must come from your group's own execution

Evidence	Must be visible / verified
Screenshot 1 - Flutter initialization and run	Project created and running with flutter run -d chrome; app shown in Chrome.
Screenshot 2 - Organized lib/ source tree	main.dart, app.dart, screens/, widgets/, services/, models/, routes/, constants/ (or actual equivalent names).
Screenshot 3 - Git and GitHub first push	Terminal Git output indicating initial commit / successful push and visible repository checkpoint details.

README.md - recommended repository documentation

Include a short README.md that agrees with your report. Another member should be able to follow the instructions and start the app without guessing the working directory or device target.

```
# Your Approved Project Title

## Summary
A short description of the community problem and first features.

## Current stack
Flutter Web / Dart; planned REST API and PostgreSQL.

## Run locally flutter pub
get flutter run -d
chrome

## First checkpoint
Date: __________  |  Commit: __________
```

7. Submission Requirements

- A single Word or PDF Activity 8 report containing the five required descriptive items, the checkpoint record, and Screenshots 1-3.
- A functioning GitHub repository link showing the initial Flutter Web source code, folder structure, and first commit.
- A project that can be opened and run locally using the documented Flutter commands.
- Submit through the instructor-designated course platform. The attached activity brief is dated October 13, 2026; follow the instructor's actual posted deadline and directions.

Academic integrity and competency validation. Screenshots, code, repository commits, and written explanations must reflect the team's actual work. AI assistance may help clarify commands or errors, but each member must understand and be able to reproduce the initialization and Git workflow.

8. Scoring Rubric - 80 Points

Award full credit when the evidence is complete, technically correct, and demonstrable. Partial credit may be given for incomplete or inconsistent output.

Criterion	Evidence required for full credit	Pts
Flutter project initialization and browser execution	Valid Flutter Web project; environment checked; successful app execution in Chrome; demonstrable first screenshot.	15

Source structure and organization	Correct, logical lib/ structure with main.dart, app.dart, starter screens, reusable widgets, and appropriate folders.	12
Initial screens and navigation baseline	Home/Login or approved project-specific starter screens exist, compile, and have a coherent navigable baseline.	12
Git/GitHub first checkpoint	Valid initial commit, correct remote and main branch, successful first push, accessible repository and verifiable history.	15
Project documentation and stack explanation	All five written items are accurate, system-specific, reproducible, and distinguish planned vs. implemented components.	10
Criterion	**Evidence required for full credit**	**Pts**
Screenshots and checkpoint evidence	All three required screenshots are genuine, readable, correctly labeled, and consistent with repository metadata.	6
Group demonstration and oral defense	Organized 5-7 minute demonstration; all three members participate, explain tasks, and answer setup/Git questions confidently.	10

Raw Score	_____ / 80	**Equivalent**	_____
Checked by	_____	**Date**	_____
Remarks	_____ _____		

9. Post-Laboratory Reflection

1. **Which error, warning, or dependency took the most time to resolve, and how did your group address it?**
   For me, one of the challenging part sa activity kay making sure nga properly configured ang Flutter environment and makadagan ang project sa Google Chrome. Nakat-on ko nga important kaayo nga i-check ang Flutter setup and dependencies before mag-start ug code. If naay errors, I would check the terminal messages and search for possible solutions para ma-fix siya. Through this activity, na-realize nako nga debugging is part of being an IT student, so need jud mag-patient and dili dayon ma-discourage.
2. **Why should main.dart/app.dart, screens, widgets, services, models, routes, and constants have separate responsibilities?**
   Para nako, it is better nga separate ang responsibilities sa each folder and file kay mas organized ug sayon sabton ang code. For example, ang `main.dart` mao ang starting point sa app, while ang `screens` kay para sa pages nga makita sa users. Ang `widgets` kay para sa reusable components, and ang `services` kay para sa future API connections. In this way, dili maglibog ang developers kung asa ibutang ang code, and mas dali i-maintain, i-debug, ug i-update ang project as the system grows.
3. **What information would a new group member need to clone this repository and run the web project successfully?**
   Kung naa koy new group member, akong ihatag niya ang GitHub repository link, the required tools like Flutter SDK, Dart, and Google Chrome, plus the instructions on how to clone and run the project. Need pud niya mahibaw-an ang correct project directory and the commands like `flutter pub get` and `flutter run -d chrome`. Para nako, helpful pud ang README.md kay naa didto ang setup instructions. This way, dili na siya maglisod ug guess sa steps and mas dali siya maka-contribute sa project.
4. **How does the initial Git commit help your group manage changes and collaboration during the semester?**

Para nako, ang initial Git commit kay murag starting checkpoint sa among project. It gives us a record sa original code and files before mi mag-add ug more features. If naay problems sa future changes, pwede namo tan-awon ang previous commits to track unsay nausab. Helpful pud ang GitHub sa collaboration kay each member can contribute code and share updates with the group. In short, mas organized among teamwork and mas dali namo ma-track ang progress sa project.

5. **Which screen or feature from your Activity 6 UI flow will your group implement first after Activity 8, and why?** For me, I would implement the **Home Screen first**, kay mao ni ang initial page nga makita sa users when they open the application. Through this screen, we can start building the basic layout and navigation before moving to other features like Login and the main system functions. Mas maayo nga magsugod mi sa simple but working nga interface para ma-test namo if okay ang structure and navigation sa app. After ana, gradually namo i-add ang other features based sa among planned UI flow.